Guarded Recursive Self-Improvement: Bounding Creeping Degradation and Reward Hacking in Autonomous Agent Loops

Abdelkabir Ouadoukou*

November 2026

si-loop

Abstract

Self-improving code agents usually follow one rule: propose a patch and keep it if a benchmark score rises. This rule fails in three ways. Individually acceptable patches can add up to a regression (*creeping degradation*). Repeated selection on the same data inflates the measured gain (*adaptive overfitting*). The agent can also learn to satisfy the checker rather than the task (*reward hacking*). We treat self-improvement as a control problem and present SI-Loop v2, a kernel that sits between a proposing agent and a sealed verifier and that the agent cannot modify. The kernel combines zero-trust git worktrees with a seven-layer patch defence, a paired non-inferiority gate applied in two stages (an anytime-valid screen on batch A, then a strict test on an independent batch B), a frozen cycle-0 anchor whose slack is limited by a margin budget that only proven gain can refill, a held-out problem family that selection never touches, a three-tier verifier with honeypots, and a hash-chained ledger with circuit breakers. The two-stage judge accepts 0.9% of null patches at its default threshold, against 4.3% for a single stage. The verifier's tiers kill 86.7% of 248 injected faults, and 99.5% after removing likely-equivalent survivors. In a pilot with 8 seeds per arm on a synthetic world, whose analysis plan was fixed before the reported runs, the gate separates informed from random patches (Holm $p = 0.0117$), but full recursion over the frozen improver is not distinguishable from zero effect, and no improver patch was accepted in any recursive run. We report these results as they came out, along with the deviations from the pre-registration. We make no claim about recursive self-improvement of language models; we provide a measurable, auditable harness in which such a claim could be tested, together with a reference implementation that includes an LLM meta-agent interface.

1 Introduction

Agents that rewrite their own code can improve. Recent systems validate self-modifications empirically [14], make the improvement procedure itself editable [15], and argue that a parent should be judged by the productivity of its descendants rather than by its own score [12]. Most of these systems rely on one signal, a benchmark score, and one rule, keeping a patch when the score rises. That rule is where the failures start.

The three failure modes above are not hypothetical. A gate that accepts any patch that is "not significantly worse" also accepts a patch that is slightly worse, and a hundred such steps can add up to a large regression. Selecting the best of many patches on one distribution inflates the gain of

*Independent researcher. Contact: abdelkabir.ouadoukou@gmail.com. Substantial AI assistance was used; see the disclosure at the end.

the winner [5]. Proxy objectives are exploited when optimised hard [3, 6], and a self-play judge can be convinced [16]. For code agents, reward hacking includes reading the tests, writing the result file, or special-casing the checker. A fourth failure is over-caution: in our earlier study [8] a guarded judge rejected about 73% of rounds, which made it look useless. Finally, a rising curve does not show that improving the *improver* helped. Without a frozen-improver control, compute-matched sampling, and random-patch baselines, "recursive" is only a label.

Contributions. This paper makes the following contributions.

- A two-stage acceptance pipeline with anytime-valid early stopping, and an A/A calibration of its false-accept rate (Section 3.2, Section 6.1).
- A margin budget and anchor audit that limit the slack a non-proven accept may spend (Section 3.4).
- A verifier design with three tiers, honeypots, and a measured mutation kill rate (Section 3.5, Section 6.2).
- A yield test for the improver, clade-based parent selection, and an operator bandit (Section 3.6).
- A tamper-evident ledger and circuit breakers for long runs (Section 3.7).
- A pre-registered ablation runner (A0–A3) and a reference implementation with hidden pytest cases and an LLM interface (Section 4, Section 6.4).

Each component has prior art. Our contribution is the integration and the measurements, reported with their limits.

2 Problem Setting and Threat Model

The meta-agent is untrusted. It may be buggy, over-optimising, or actively trying to raise its score by illegitimate means. It can read its own worktree and has a token budget. It submits at most one structured patch per cycle. We do *not* defend against code execution outside the sandbox, against access to the verifier's secret, or against bugs in the kernel. Local mode is not a security boundary.

The design follows six principles. (P1) The judge is not part of what is judged, and a digest check aborts the run if the kernel changes. (P2) Agents propose and the kernel disposes: there are no direct writes. (P3) Every decision uses fresh data, because problem identifiers are HMACs of a secret and are drawn per split and per cycle. (P4) Feedback to the agent is coarse: accept, stepping stone, or reject, with a brief error type. (P5) Every non-proven accept spends from a finite budget, and audits compare the champion with a frozen reference. (P6) Recursion is measured against controls, not against nothing.

3 The SI-Loop v2 Architecture

Figure 1 shows the trust boundaries. The kernel orchestrates each cycle: it creates a worktree from a chosen parent, asks the meta-agent for a proposal, checks it, runs the two-stage judge, and promotes or rolls back. The agent never sees hidden tests, expected values, problem content, per-family numbers, pass counts, audit results, or held-out results.

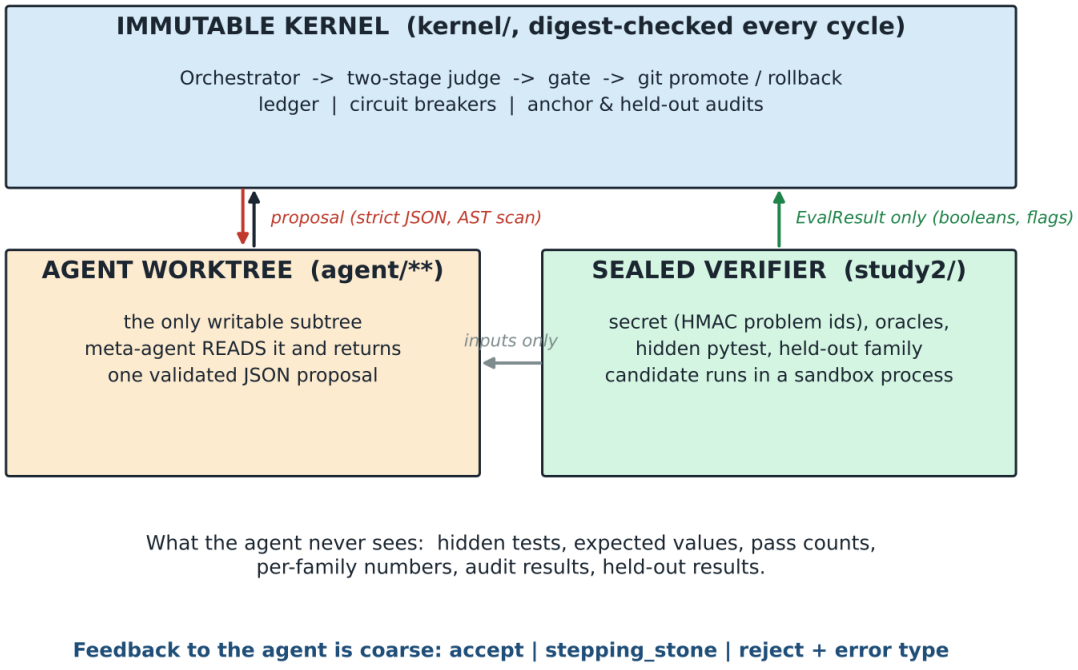


Figure 1: Trust boundaries. Arrows show the only data that cross them: a validated proposal goes down, and aggregate booleans and flags come back up.

3.1 Zero-trust worktrees and seven-layer patch defence

Each cycle works in a git worktree. A proposal is a list of at most eight hunks, and each hunk is a unit of credit assignment. The kernel checks every proposal through seven layers before any test runs (Table 1). The AST scan is a first filter, not a security boundary. Real isolation comes from the sandbox process: resource limits, a scrubbed environment, no network, and process-group kill. A container mode adds a read-only root and `-network none`, but it is untested here.

3.2 The paired non-inferiority gate

Parent and candidate run on the same fresh problems. Let b be the number of problems the patch fixed, c the number it broke, and n the number of problems. Define $\delta = (b - c)/n$, $\mathrm{se} = \sqrt{b + c - (b - c)^2/n}\,/n$, and the one-sided lower bound $\ell = \delta - z\,\mathrm{se}$ with $z = 1.645$. This is a normal approximation, the standard form of a non-inferiority bound [1]. With a flat prior on the fix probability among discordant pairs,

$$p_{\text{better}} = P\big(\mathrm{Bin}(b + c + 1, \tfrac{1}{2}) \le b\big).$$

A patch is accepted if $\ell \ge -\varepsilon$, $\delta > 0$, and $p_{\text{better}} \ge p_{\min}$. It is also accepted on the *utility path* if it is non-inferior and clearly smaller or cheaper, which spends from the margin budget (Section 3.4). A non-inferior patch that is behaviourally novel becomes a *stepping stone*: it is archived, not promoted. Novelty can never rescue a regression. The threshold $p_{\min}$ is chosen by A/A calibration for the batch size at hand (Section 6.1).

Table 1: The seven layers applied to every proposal, in the order they run.

#	Layer	What it rejects
1	Strict parse	unknown keys, wrong types, more than eight ops, oversized payloads, bad tags
2	Path policy	anything outside `agent/`, traversal, dotfiles, `conftest.py`, other suffixes
3	AST scan	forbidden imports (`socket`, `subprocess`, `ctypes`, ...), `eval`, `exec`, `open`, dunder escapes, dangerous `os.*`
4	Test ratchet	removed tests; `skip`, `xfail`, `importorskip`
5	Atomic exact apply	ambiguous symbols; a failing op leaves the tree unchanged
6	Diff audit	committed files that differ from the files named in the proposal
7	Sandboxed self-tests	failing public tests, timeouts, memory blow-up

3.3 Two stages, sequential stopping, and online FDR

The screen runs on batch A with a lenient threshold $p_{\text{screen}} = 0.80$, in chunks. A candidate that passes is re-evaluated on an *independent* batch B, and the strict gate is applied to B alone. B was not used to select the candidate, so the winner's curse does not apply to it. Because both stages must pass, the false-accept probabilities multiply.

After each chunk of A we update the discordant counts. For harm we use the e-value

$$E_{\downarrow} = B(b{+}1, c{+}1) \sum_{k \le c} \binom{b+c+1}{k},$$

the mirror image of the improvement e-value $E_{\uparrow} = B(b{+}1, c{+}1) \sum_{k \le b} \binom{b+c+1}{k}$. Each is a mixture over $q \sim U(\frac{1}{2}, 1)$ of a likelihood ratio, and so is an e-value under every null with fix probability at most $\frac{1}{2}$. By Ville's inequality [9], stopping at any data-dependent time keeps the error bound. For futility we use Wald's sequential probability ratio test [11] with alternative $p_1 = \frac{1}{2} + \theta$. A stop only drops a candidate early. It cannot create an accept, because acceptance happens on B.

Over thousands of cycles the number of null accepts grows. Confirmed accepts can be passed to e-LOND [13], which controls the false discovery rate online under arbitrary dependence. It runs in monitor mode (logged) or enforce mode (blocking).

3.4 Anchor, margin budget, and audits

The champion at cycle 0 is frozen as the *anchor*. Utility-path accepts spend from a finite margin budget B_0. The charge of an accept with lower bound ℓ_i is $c_i = \max(\varepsilon_{\min}, -\ell_i)$, which is the amount of regression its bound could hide, with a floor. The budget is refilled only by a *new high-water mark* of proven gain over the anchor, measured on fresh batches. Every K cycles an audit compares the champion with the anchor and with its own historical peak, and rolls back on regression.

Proposition 1 (Budget bound; sketch)**.** *Suppose utility-path accepts $i = 1, \dots, k$ have true paired changes δ_i and lower bounds ℓ_i with $P(\delta_i \ge \ell_i) \ge 1 - \alpha$, and charges $c_i \ge \max(0, -\ell_i)$. On the event that all bounds hold, which has probability at least $1 - k\alpha$ by a union bound, the total regression introduced by these accepts satisfies $\sum_i \max(0, -\delta_i) \le \sum_i c_i \le B_0 + R$, where R is the total refill.*

Sketch. On the event, $-\delta_i \le -\ell_i \le c_i$ whenever $\delta_i < 0$. Spending cannot exceed the balance, so $\sum_i c_i \le B_0 + R$. □

The proposition bounds the sum of step losses, not the net drift. It assumes valid per-step bounds (ours are normal approximations) and ignores correlation between steps. The anchor audit is the backstop for exactly these gaps, and Section 6.4 shows a case where the bound was exceeded on a final batch.

3.5 Held-out family, verifier, and mutation testing

One problem family is drawn *only* by the audit split. It is never used for selection, confirmation, credit, anchor, or yield batches. The cross-family gap is the selection gain minus the held-out gain of the champion over the anchor. A large gap pauses the run. Audit decisions that read reusable data use a Thresholdout-style noisy comparison [5], and results go to the ledger, never to the agent.

Each problem is checked by three tiers: small-scope exhaustive inputs, random large-scope fuzzing, and metamorphic relations (for example, adding spaces around operators must not change an arithmetic result). Honeypots are impossible problems with junk expectations that exist only in decoy files. Passing one, opening a decoy path, or emitting a canary sets a security flag, causes an automatic reject, and quarantines the commit. To measure the verifier, we apply AST mutations to known-correct reference solutions and count how many mutants the tiers reject [4, 7].

3.6 Yield testing, clades, and the operator bandit

A patch that edits the improver (`agent/meta/**`) changes how future patches are made, so a solver score cannot judge it. We measure its *yield*: the gain in pass rate after K inner cycles on a fresh batch, from identical starting states and seeds. The candidate improver, its parent, and the frozen reference improver M0 are all evaluated this way. The patch is accepted only if the bootstrap lower bound of the paired yield difference to the parent is positive and the yield is not worse than M0 by more than a tolerance. Patches that mix improver and solver edits are rejected. Parents are chosen by Thompson sampling on discounted descendant successes, following the clade idea of Wang et al. [12], and an operator bandit keeps an exponential moving average of reward for each hunk tag.

3.7 Long-run autonomy

The ledger is append-only and hash-chained, with $h_t = \mathrm{SHA256}(h_{t-1} \,\|\, \mathrm{canon}(r_t))$, so any edit to history is detected on verification. Circuit breakers trip on accept-rate anomalies (a binomial tail against a pre-registered rate), LOC explosions, cost spikes, repeated sandbox-escape attempts, security flags, and repeated audit regressions. A trip pauses the run or latches a HALT file. A heartbeat and an external watchdog cover silent crashes. A model and kernel fingerprint is logged, and a change halts the run because it invalidates calibration. Orphan worktrees and stale branches are garbage-collected.

4 Reference Implementation

Earlier drafts had only interfaces for the verifier and the meta-agent. The repository now contains a complete reference pair.

Task suite and hidden pytest. The `study2/` package defines 15 tasks in three families: string (6), algorithmic (6), and a held-out parsing family (3). Each task has a public specification (a docstring), a judge-side oracle, and a case generator that mixes ordinary and edge-case inputs. The seed agent has about half of its functions buggy or missing. For each evaluation the judge derives

the tasks from HMAC-based identifiers, copies the candidate's `agent/` into a sandbox process that receives inputs only, and writes its outputs to a result file tagged with a per-run nonce. The judge then writes a hidden pytest file with expected and actual values, and runs pytest in its own process. A problem passes if its test passes on all six cases. Candidate code never shares a process with pytest or an oracle, and a result without the correct nonce is ignored. A separate self-check confirms that a library of correct solutions passes 100% of problems, so there are no false rejects.

LLM meta-agent. `LLMMetaAgent` builds a prompt from the agent's source and from untrusted lessons, requests a JSON proposal, parses it with the kernel's strict `parse_proposal`, and retries malformed replies with the error text. All calls are charged to the cycle's token budget. Clients exist for any OpenAI-compatible endpoint (a local Ollama, vLLM, llama.cpp, or LM Studio server, or OpenAI), for the Anthropic API, and for a scripted offline client that is used for tests and demos. `run_study2.py` runs the `check`, `demo`, `run`, and `report` commands.

5 Experimental Setup

We separate what is measured from what is demonstrated. Sections 6.1 and 6.2 measure components. Section 6.3 demonstrates the pipeline with a scripted proposer. Section 6.4 reports a pre-registered pilot on a synthetic world. All numbers come from files in `results/study2/` and are inserted by `make_paper_numbers.py`. Appendix A gives the pre-registration timeline and the deviations from it.

Ablation arms. The unit of analysis is the run (seed), and the seed is paired across arms. The four arms are: A0, a frozen improver that makes solver patches only; A1, full recursion, in which the improver is judged by yield tests; A2, a random-patch control that passes through the same gate; and A3, a compute-matched best-of-N, in which every proposal is made from the anchor and the champion is the best single patch. Each run lasts 12 cycles in the smoke profile on 8 seeds. The final evaluation uses fresh batches shared by every arm and seed.

Hypotheses and tests. H1 says that A1 beats A0 on the final selection-family gain. H2 says that A0 beats A2, and H3 says that A0 beats A3. Each is a one-sided paired sign-flip test, with Holm correction across H1–H3. H4 says that A1 is not worse than A0 by more than 0.02 on the held-out gain. Bootstrap confidence intervals are reported for each paired difference. With 8 seeds, the smallest attainable one-sided p-value is 0.0039.

6 Results

6.1 False-accept calibration and power

We simulate A/A tests. Both versions have the same true pass rate $p = 0.5$, with independent run-to-run noise of $\sigma = 0.1$ on the latent scale, with $n = 200$ problems per batch and 2000 trials per setting. At $p_{\min} = 0.95$ the single-stage gate accepts 4.3% of null patches and the two-stage judge accepts 0.9% (Table 2, Figure 2). The calibration routine would choose 0.80 as the smallest threshold that meets a 5% target for the two-stage judge. We keep 0.95 as the default because the real run-to-run noise is unknown, and a margin is cheaper than a false accept. The cost is power. For a patch that truly adds 2, 4, 6, and 8 points of pass rate, the single stage accepts 22%, 52%, 80%, and 95% of patches, and the two-stage judge accepts 12%, 40%, 75%, and 94%. Small

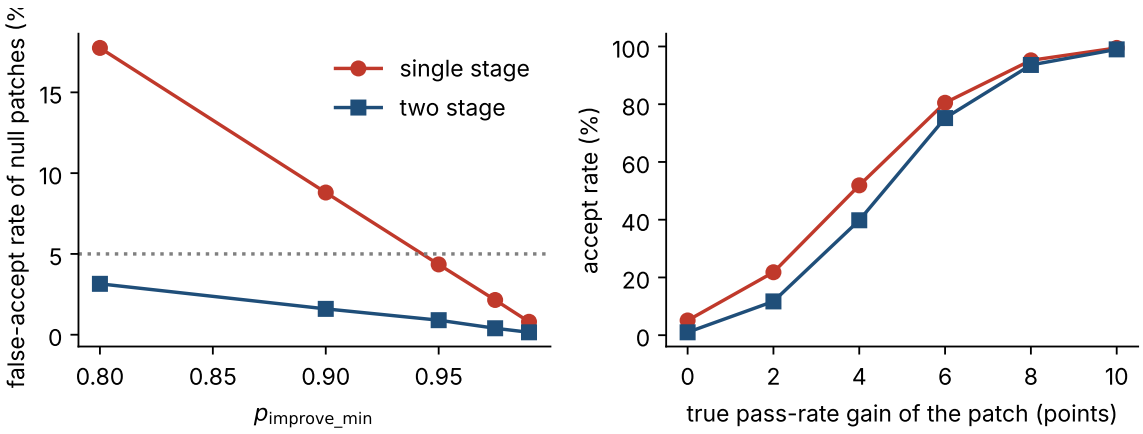


Figure 2: Left: false-accept rate of null patches as a function of $p_{\text{improve_min}}$, for a single stage and for the two-stage judge (dotted line: 5%). Right: accept rate as a function of the true pass-rate gain, at $p_{\text{improve_min}} = 0.95$.

Table 2: A/A false-accept rate of null patches ($n = 200$, $p = 0.5$, $\sigma = 0.1$, 2000 trials). The two-stage judge is the default.

$p_{\text{improve_min}}$	single stage	two stage
0.800	17.8%	3.1%
0.900	8.8%	1.6%
0.950	4.3%	0.9%
0.975	2.1%	0.4%
0.990	0.8%	0.1%

true gains are the ones that are lost. They are also the ones that noise most easily produces. The noise model is an assumption. Run-to-run noise must be estimated from repeated evaluations of an unchanged agent before these numbers are used.

6.2 Verifier quality

On the synthetic world's nine sealed task kinds we generated 248 mutants of the reference solutions. The three tiers killed 215 of them, a raw kill rate of 86.7%. Of the survivors, 32 also passed an extra-large fuzz run, so we treat them as likely equivalent mutants. Removing them gives 99.5%. This label is a heuristic, not a proof, so the raw rate is the conservative figure. Kills by tier were small-scope 203, fuzz 211, and metamorphic 23. The mutants killed *only* by one tier were 4 (small-scope), 12 (fuzz), and 0 (metamorphic). The metamorphic tier therefore added no kill that the other tiers missed in this suite. The correct reference solutions failed 0 times, and the planted snooping solver was detected: yes. These numbers measure sensitivity to injected faults in the synthetic world. They do not bound the probability of a semantic loophole that an LLM could find, and they were not measured on the `study2/` tasks.

6.3 End-to-end pipeline run with a scripted proposer

`run_study2.py demo` runs 14 cycles of the reference implementation with a *scripted* proposer. The script contains a list of fixes, one deliberate regression, and one malformed reply. It is not a

Table 3: Arms, mean ± standard deviation over 8 seeds. Gain is the final pass rate minus the anchor's, on batches shared by all arms. Tokens include yield tests.

Arm	seeds	gain, selection families	gain, held-out	accepts	tokens
A0	8	$+0.178 \pm 0.129$	$+0.000$	1.5	12,534
A1	8	$+0.302 \pm 0.220$	$+0.000$	1.8	29,098
A2	8	-0.017 ± 0.018	$+0.000$	2.6	11,776
A3	8	$+0.242 \pm 0.122$	$+0.000$	1.0	12,429

Table 4: Pre-registered contrasts, paired by seed. The p-values are one-sided sign-flip tests. Holm correction is applied across H1–H3. The confidence intervals are 95% bootstrap intervals of the mean paired difference.

Hyp.	contrast	pairs	mean diff.	95% bootstrap CI	p (one-sided)	Holm p
H1	A1−A0	8	$+0.124$	$[-0.076, +0.319]$	0.1523	0.3047
H2	A0−A2	8	$+0.195$	$[+0.120, +0.272]$	0.0039	0.0117
H3	A0−A3	8	-0.064	$[-0.177, +0.066]$	0.8438	0.8438

language model, and the run shows the machinery only. Of the 14 cycles, 6 were accepted, 7 were rejected, and the remaining 1 was archived as a stepping stone. The hash chain of the ledger was verified. On fresh batches the champion's pass rate on the selection families rose from 0.28 (anchor) to 0.78, with 60 problems fixed and 0 broken out of 120. The held-out family stayed at 0.38 for both anchor and champion, so the cross-family gap was 0.50, and the generalisation alarm fired. This is the intended behaviour: the scripted proposer only fixes functions in the selection families. Of the rejections, 3 passed the screen on batch A but failed the confirmation on batch B, which is the winner's curse being removed in practice. Proposals that changed no code are rejected as parse errors; the scripted list repeats some fixes, which gives 2 such rejections. The malformed reply was retried before it reached the gate. The deliberate regression (a function that returns zero) was caught by the sandboxed self-tests (1 self-test rejections). The run uses a fixed, public demo secret, so it reproduces exactly; the reported numbers come from that run.

6.4 Pre-registered ablation pilot

Table 3 lists the arms and Table 4 the contrasts. No run halted, tripped a security flag, or broke its ledger (0 such runs out of 32). The mean final gain on the selection families over the frozen anchor was +0.178 for A0, +0.302 for A1, -0.017 for A2, and +0.242 for A3.

Figure. Figure 3 shows every seed as a dot, which the table above cannot show.

Results & discussion. H2 is supported. Random patches behind the same gate did not improve the agent (mean -0.017), whereas informed proposals behind the same gate did (Holm $p = 0.0117$). So the gain does not come from the gate alone. H1 is not supported. The point estimate favours A1 (mean difference 0.124, 95% CI [-0.076, 0.319], Holm $p = 0.3047$), but the interval contains zero. More importantly, no improver patch was accepted in any A1 run: 13 improver patches were proposed across the eight runs, and 0 were accepted. A1 therefore ran essentially the same policy as A0, except that about 15% of its proposals were spent on edits that were rejected. The positive difference reflects run-to-run variation in the proposal stream, which varies widely across seeds. It is not evidence of recursion. H3 is not supported (mean -0.064, Holm $p = 0.8438$). Compute-matched

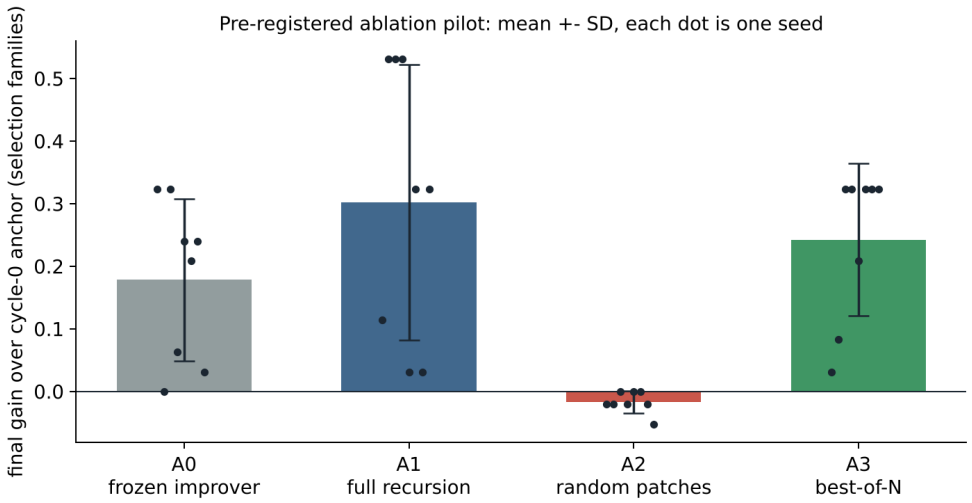


Figure 3: Final gain over the cycle-0 anchor on the selection families, by arm. Bars are means, error bars are standard deviations, and each dot is one seed (8 per arm).

best-of-N from the anchor was at least as good as iterative accumulation. In this small world each useful fix is independent, so iteration adds nothing over sampling. A1 used 2.3 times the tokens of the other arms, because of the yield tests. A1 against A3 was not pre-registered and is not tested. H4 is uninformative. The held-out gain was +0.000 in every arm, because the toy improver never edits the interpreter family and the margin holds trivially. The held-out audit machinery is exercised by the end-to-end run in Section 6.3, but this ablation says nothing about cross-family overfitting.

The random arm and the margin budget. A2 accepted 2.6 patches per run on average. Its final gain ranged down to -0.052 in the worst run. The margin spent per run ranged from 0.005 to 0.020, against a budget of 0.03. The worst run therefore lost -0.052 on the final batch while spending only 0.005 of budget. This is a counter-example to reading the budget as a hard bound on true loss. The charge is based on a lower bound, the final batch of 96 problems is itself noisy, and Proposition 1 holds only when the per-step bounds hold. What the data support is narrower. With the budget and floor in place, a policy that proposes random edits lost about two points on average, reached a worst final gain of -0.052, and never gained over the anchor on average.

7 Discussion and Limitations

Adaptive overfitting. The two-stage design removes the winner's curse for each decision, and fresh HMAC batches prevent reuse. Over many cycles, though, an agent can still adapt to the *distribution* (families, edge-case style) through the code it keeps. Coarse feedback limits this leakage but does not eliminate it. The held-out family is the detector, but it only detects failures to generalise across the families we thought of. Whether an LLM would overfit in this way is an open empirical question.

Verifier quality. Mutation kill rate measures sensitivity to simple injected faults. It says little about semantic loopholes, such as an under-specified oracle or a missing edge-case family. Honey-

pots detect curiosity, not stealth. A verifier should be red-teamed by an adversarial model before anyone claims "no reward hacking". We have not done that.

Margin budget. Proposition 1 is conditional and bounds a sum, not drift. Charges can be underestimated, and the observed run above shows it. The budget makes creep expensive and visible, and the audit makes it reversible, but neither makes it impossible.

From a synthetic mutation engine to a real LLM. The ablation uses a simulated improver because it is deterministic, free, and fast. Its dynamics (a handful of operators and a library of fixes) are far from an LLM's. The step to a model is where most uncertainty lies. Proposals become far more varied, token costs matter, prompt injection through source comments becomes a failure mode, and run-to-run noise rises. Batch sizes and $p_{\min}$ must then be recalibrated. The kernel does not change. The calibration, the budgets, the number of seeds, and the cost of yield tests do. We provide the interface and a local-model path but report no LLM results.

Other limits. The main limits are these. (i) The pilot has 8 seeds, 12 cycles per run in the smoke profile, a very small yield test, and a small synthetic world. Its confidence intervals are wide. (ii) The pre-registration was locked after two aborted pilots, and it was not committed to git before the runs. Appendix A lists what changed. (iii) Container mode is untested. The local sandbox is not a security boundary. (iv) Costs are counted in lines of code and tokens, not deterministic instruction counts. (v) The normal-approximation lower bound is not exact for small discordant counts. (vi) Several references are marked FROM-KNOWLEDGE in `references.bib` and must be checked before submission.

8 Related Work

Self-improving agents. The Darwin Gödel Machine [14] and hyperagents [15] rewrite agents and their improvement procedures with empirical validation. The Huxley-Gödel Machine [12] scores parents by the productivity of their descendants, and our clade-based selection follows that idea. Gödel machines [10] give the classical formal framework. Surveys cover the wider field [2]. Our difference is the guard: a kernel, a calibrated gate, and controls that these systems do not isolate.

Overfitting and reward hacking. Reusable holdout data and adaptive analysis are studied in Dwork et al. [5]. Proxy overoptimisation is documented by Gao et al. [6] and Coste et al. [3], and the vulnerability of self-play judges by Zhou [16]. Our approach is to make the judge fresh, coarse, and sealed, and to measure the gap that remains.

Sequential and anytime-valid inference. E-values and game-theoretic tests [9] justify stopping at any time. e-LOND [13] controls false discoveries online. The sequential probability ratio test [11] is the classical tool for futility stopping.

Mutation testing. The kill rate of injected faults as a measure of test adequacy goes back to DeMillo et al. [4]. A survey is given by Jia and Harman [7].

9 Conclusion

Guarding self-improvement is mostly bookkeeping. One must decide who may write, which data decide, how much slack remains, and which control each measurement is compared against. SI-Loop v2 puts that bookkeeping in an immutable kernel, calibrates its acceptance rule, and ships a pre-registered harness for testing whether recursion helps. In our pilot the harness behaved as designed: the gate separated informed from random search, and the recursive arm produced no accepted improver patches. The next step is the one this paper does not take: running the harness with a real LLM meta-agent, with the seeds and calibration that such a setting requires.

Reproducibility. Code and data: https://github.com/abdelkabirouadoukou/si-loop. Tests: `python -m pytest -q tests`. Verifier self-check: `python run_study2.py check`. Demo: `python run_study2.py demo`. Calibration: `python results/study2/make_calibration.py`. Verifier quality: `python -m kernel.mutation`. Ablations: `python run_ablations.py prereg|run|analyze`. All numbers are generated by `results/study2/make_paper_numbers.py` from the raw files in `results/study2/`. Study 1 is frozen and unchanged; `si_loop_toy.py` is pinned by hash in `results/config_hashes.txt`.

Disclosure. Substantial AI assistance was used to write the code and to draft this report. The author chose the design and is responsible for the content. Numbers come from generated files. References marked FROM-KNOWLEDGE were not re-fetched.

References

[1] William C. Blackwelder. "proving the null hypothesis" in clinical trials. *Controlled Clinical Trials*, 3(4):345–353, 1982.

[2] Mingguang Chen, Licheng Wang, and Bo Qu. Recursive self-improvement in AI: From bounded self-refinement to autonomous research loops. *arXiv preprint arXiv:2607.07663*, 2026.

[3] Thomas Coste, Usman Anwar, Robert Kirk, and David Krueger. Reward model ensembles help mitigate overoptimization. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.02743.

[4] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4):34–41, 1978.

[5] Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth. The reusable holdout: Preserving validity in adaptive data analysis. *Science*, 349(6248): 636–638, 2015.

[6] Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. arXiv:2210.10760.

[7] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011.

[8] Abdelkabir Ouadoukou. A controlled ablation study of a guarded self-improvement loop with a competitive global workspace in a toy setting. Technical report, Independent researcher, 2026. Study 1 of this repository; not peer reviewed.

[9] Aaditya Ramdas, Peter Grünwald, Vladimir Vovk, and Glenn Shafer. Game-theoretic statistics and safe anytime-valid inference. *Statistical Science*, 38(4):576–601, 2023.

[10] Jürgen Schmidhuber. Gödel machines: Fully self-referential optimal universal self-improvers. *Artificial General Intelligence (Cognitive Technologies), Springer*, 2007. arXiv:cs/0309048.

[11] Abraham Wald. Sequential tests of statistical hypotheses. *Annals of Mathematical Statistics*, 16(2):117–186, 1945.

[12] Wenyi Wang, Piotr Piękos, Li Nanbo, Firas Laakom, Yimeng Chen, Mateusz Ostaszewski, Mingchen Zhuge, and Jürgen Schmidhuber. Huxley-Gödel machine: Human-level coding agent development by an approximation of the optimal self-improving machine. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.21614.

[13] Ziyu Xu and Aaditya Ramdas. Online multiple testing with e-values. In *Proceedings of the 27th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 238 of *PMLR*, 2024.

[14] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin Gödel machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*, 2025.

[15] Jenny Zhang, Bingchen Zhao, Wannan Yang, Jakob Foerster, Jeff Clune, Minqi Jiang, Sam Devlin, and Tatiana Shavrina. Hyperagents. *arXiv preprint arXiv:2603.19461*, 2026.

[16] Chenyu Zhou. More convincing, not more correct: Self-play reward hacking of reference-free LLM judges. *arXiv preprint arXiv:2607.05904*, 2026.

A Pre-registration timeline and deviations

The pre-registration file `results/study2/ablation/prereg.json` was written before the reported runs and locked to the kernel digest (hash prefix b36d851de8d8). Its hash is checked before every run. Before it, two pilot ablations were started and stopped. They used 24 cycles with 8 seeds, and then 12 cycles with 4 seeds. Both were stopped because they were too slow for the available machine. The reported configuration was chosen after these pilots, so the pre-registration is not independent of them. The pre-registration was not committed to git before the runs. The analysis rules in `run_ablations.py` were not changed after the results were seen. The yield test used one start, two seeds, and $k = 2$ inner cycles, which is much smaller than the full setting.

B Reproduction

```
python -m pytest -q tests                      # 80 tests
python run_study2.py check                     # verifier self-test
python run_study2.py demo --cycles 14 --n 48  # scripted end-to-end run
python results/study2/make_calibration.py      # calibration and power curves
python -m kernel.mutation --per-kind 15 --instances 2 --out results/study2/verifier_quality.jso
python run_ablations.py prereg --out runs/abl --arms A0 A1 A2 A3 --seeds 8 --cycles 12 --profil
python run_ablations.py run --out runs/abl --jobs 2
python run_ablations.py analyze --out runs/abl
python results/study2/make_paper_numbers.py
```